

CSA0619-DESIGN ANALYSIS OF ALGORITHM

CO4 AT3 ANSWERS

BY ANTONY MAGRIN J(192572103)

Q3. MST Comparison — Prim's vs Kruskal's Algorithm

1. Problem Definition

Input: a connected, weighted, undirected graph — n vertices, edges = $[(u, v, \text{weight}), \dots]$

Output: a Minimum Spanning Tree — $n-1$ edges connecting all vertices, no cycles, minimum total weight — computed independently by both algorithms for comparison.

2. Pseudocode

ALGORITHM Prim(graph, n, source)

ALGORITHM Kruskal(edges, n)

INPUT : adjacency list, n vertices, start node

INPUT : edge list (u,v,w), n vertices

OUTPUT: MST edge set, total weight

OUTPUT: MST edge set, total weight

1 visited <- {source}

1 SORT edges by weight ascending

2 MST <- {}

2 MST <- {}

3 minHeap <- all edges from source, keyed by w

3 initialize Union-Find over n vertices

4 WHILE minHeap not empty AND |visited| < n:
edges:

4 FOR each edge (u, v, w) in sorted

5 (w, u, v) <- extractMin(minHeap)

5 IF find(u) != find(v): // no cycle

6 IF v in visited: CONTINUE // stale entry

6 MST.add((u, v, w))

7 visited.add(v)

7 union(u, v)

8 MST.add((u, v, w))

8 IF |MST| == n-1: BREAK

9 FOR each (v, x, w2) in graph[v]:

9 RETURN MST

10 IF x not in visited:

```

11         push (w2, v, x) to minHeap
12 RETURN MST

```

3. Edge Selection Strategy — Key Difference

	Prim's	Kruskal's
Approach	Vertex-centric — grows one connected tree outward	Edge-centric — considers all edges globally , sorted by weight
Choice rule	Always add the cheapest edge leaving the current tree to a new vertex	Add the next-cheapest edge in the whole graph if it doesn't create a cycle
Data structure	Min-heap (priority queue) over frontier edges	Sorted edge list + Union-Find (disjoint set)
Intermediate state	Always ONE growing tree	May be a forest of several fragments that merge over time
Cycle avoidance	Implicit — only considers edges to unvisited vertices	Explicit — $\text{find}(u) \neq \text{find}(v)$ check via Union-Find

4. Side-by-Side Trace (6-vertex sample graph)

Edges: (0,1,4) (0,2,4) (1,2,2) (2,3,3) (2,5,2) (2,4,4) (3,4,3) (5,4,3)

PRIM (from vertex 0):

```

ADD (0-1, w=4) -> grows tree to include 1
ADD (1-2, w=2) -> grows tree to include 2
ADD (2-5, w=2) -> grows tree to include 5
ADD (2-3, w=3) -> grows tree to include 3
ADD (3-4, w=3) -> grows tree to include 4
Total weight: 14

```

KRUSKAL (sorted edges, Union-Find):

```

ADD (1-2, w=2) -> merges two components
ADD (2-5, w=2) -> merges two components
ADD (2-3, w=3) -> merges two components

```

ADD (3-4, w=3) -> merges two components

reject (5-4, w=3) -> would form a cycle

ADD (0-1, w=4) -> merges two components

Total weight: 14

Both reach the **same total weight (14)**, confirming correctness — the edge sets can differ slightly when weights tie, but both remain valid minimum spanning trees. Notice how Prim's picks always touch the growing frontier, while Kruskal's can merge two far-apart fragments in one step (e.g. connecting $\{1,2,5,3,4\}$ to $\{0\}$ only at the very end).

5. Performance — Sparse vs Dense Graphs

Type	V	E (approx)	Prim time	Kruskal time	Same weight?
Sparse	200	398	0.00034s	0.00031s	True
Sparse	500	1746	0.00235s	0.00122s	True
Dense	200	10149	0.00482s	0.00183s	True
Dense	500	62874	0.06809s	0.01287s	True
Dense	500	112774	0.11167s	0.01645s	True

- **Sparse graphs:** the two are close, with Kruskal often slightly faster since $E \log E$ sorting on few edges is cheap and Union-Find operations are near $O(1)$ (with path compression).
- **Dense graphs:** Kruskal clearly outpaces Prim in this Python implementation, since sorting $E \log E$ on a large but not-huge edge set is efficient, while Prim's heap accumulates many stale/duplicate entries ($O(E \log V)$ with a binary heap, but constant factors from heap churn grow with density).
- **Theoretical complexity** (with a binary heap for Prim, adjacency list): Prim = $O(E \log V)$, Kruskal = $O(E \log E)$ (dominated by the sort). Since $\log E \approx \log V^2 = 2 \log V$ for dense graphs, both are asymptotically close; Kruskal's simpler Union-Find bookkeeping tends to win in practice unless a Fibonacci-heap Prim ($O(E + V \log V)$) is used, which is rarely implemented in production code due to complexity

.

6. Implementation Differences

- **Prim's** needs an adjacency-list representation and a priority queue; naturally suited to incremental, online growth (useful if the graph is discovered edge-by-edge, e.g. streaming network topology).
- **Kruskal's** needs only a flat edge list plus Union-Find; simpler to implement and parallelize (sorting can be parallelized; Union-Find is close to $O(\alpha(n))$ per operation, i.e. practically constant).
- Kruskal's naturally reports the sequence of component merges, useful if you want to see connectivity build up (e.g. clustering, network segmentation analysis).

7. Conclusions on Suitability

- **Dense graphs ($E \approx V^2$):** Prim's with an adjacency-matrix and simple array-based minimum search ($O(V^2)$) is traditionally preferred, since it avoids the overhead of managing a huge heap; but with a heap-based Prim as tested here, Kruskal's tends to be faster in practice due to simpler per-edge overhead.
- **Sparse graphs ($E \approx V$):** Kruskal's $O(E \log E)$ and Prim's $O(E \log V)$ are both efficient and close in performance; Kruskal's is often preferred for its simplicity.
- **Streaming/incremental graphs:** Prim's is more natural since it grows the tree vertex-by-vertex without needing the full edge list upfront.
- **Distributed/parallel systems:** Kruskal's decomposes more easily (parallel sort + union-find), making it attractive at scale.